Learning Objectives

This lab lets you practice nested loops (including branching in the loops), and nested loops.

Estimated Size

1 program file: `nested_loops.py`, 2 functions, and around 15-30 lines of code in each function.

Setup

In your Python files, you must include author information. On the first line, add a **`# Author`** comment line, where you mention your full name. Similarly, include your email and Spire ID on the **`# Email`** and **`# Spire ID`** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

0. nested loop basics

A nested loop is a loop in the body of another loop. We refer to them as the **inner loop** and the **outer loop**. It sounds complicated but don't be scared! On page 2 of this document, the example of calculating how many primes there are between `[a, b ]` is intrinsically a form of nested loop. The outer loop iterates over all integers between `[a, b]`, and the inner loop is in the `is_prime` function, which you wrote in lab 5 using a `while` loop. So this is a form of a loop inside another loop -- a nested loop.

The simplest nested loop is one where the inner loop iterations do not depend on the outer loop iterations. For example, the code below prints out a multiplication table. The outer loop runs over numbers from `1` to `9`. For each outer loop iteration, the inner loop also runs over `1` through `9`, independent of what the outer loop variable `i` is. So in total the inner loop body will run `9x9=81` times. By default, the `print` function will end the print with a newline. The `end=' '` argument tells it to end the print with a space character instead of a newline.

```
for i in range(1, 10):                  # outer loop iterates over [1, 9]
  for j in range(1, 10):                # inner loop iterates over [1, 9]
    print(f'{i}x{j}={i*j}', end=' ')    # inner loop body, print multiplication
  print()                               # print a new line
```

1. Implement the function `get_names` (8 points)

Your first exercise is to write a function called `get_names` which takes two lists of strings: one representing a list of first names (given names), the other a list of last names (surnames), and the

function returns a new list that contains **all combinations** of first names and last names. For example, if

```
first_names = ['Ari', 'Taylor']
last_names = ['Levine', 'Lopez', 'Khan', 'Wang']
```

Your function should return a new list that contains 8 full names:

```
['Ari Levine', 'Ari Lopez', 'Ari Khan', 'Ari Wang', 'Taylor Levine',
 'Taylor Lopez', 'Taylor Khan', 'Taylor Wang']
```

Specifically, write the function in the same file as before, and it should:

- Take two lists of strings as parameters, one `first_names`, the other `last_names`
- Use `full_names=[]` to create an empty list named `full_names`.
- Use nested `for` loops to iterate over the lists of first names and last names. For each combination of first name and last name, generate the full name, and append that to the `full_names` list. Each full name **must have a space character between** the first name and the last name.
- After the loop, return the `full_names` list.
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function.

2. Implement the function `average_scores` (7 points)

In this exercise, you will write a function called `average_scores` that calculates the average grade for each student after applying a lateness penalty. The input is a list of lists, where each inner list represents a student's assignments. Each assignment is represented as a tuple of (grade, lateness). Grades are between 0 and 100 inclusive. For example:

```
[[(90, 0), (80, 1), (70, 2), (60, 3), (50, 4)], [(100, 10), (90, 0), (80, 0), (90, 0)], [(0, 0), (20, 0), (40, 1), (100, 0), (100, 0), (100, 0)]]
```

As you can see, it's a list of lists of tuples: the size of the top-level list is the number of students (so there are 3 students in the example above) where each sub-list contains assignments for a student. Each assignment is represented as a 2-element tuple in a format of (raw grade, lateness). Specifically, your function should:

- Take the input described above as the parameter.
- Apply the lateness penalty as follows:
  - Lateness = 0 → 100% credit (no penalty)
  - Lateness = 1 → 90% credit
  - Lateness = 2 → 75% credit
  - Lateness = 3 → 50% credit
  - Lateness ≥ 4 → 0% credit
- Return a **list** where each element is the average grade of each student after applying penalties.
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function.

```
scores = [[(90, 0), (80, 1), (70, 2), (60, 3), (50, 4)],
          [(100, 10), (90, 0), (80, 0), (90, 0)],
          [(0, 0), (20, 0), (40, 1), (100, 0), (100, 0), (100, 0)]]
```

```
print(average_scores(scores))
```

```
Output → [48.9, 65.0, 59.333333333333336]
```

```
Explanation:
For the first student: (90 + 80 * 0.9 + 70 * 0.75 + 60 * 0.5 + 50 * 0) / 5 = 48.9
For the second student: (100 * 0 + 90 + 80 + 90) / 4 = 65.0
For the third student: (0 + 20 + 40 * 0.9 + 100 + 100 + 100) / 6 = 59.3333
```

3. Submit your code to Gradescope

Log in to Gradescope, select **Lab 06: (Code)**, and submit the required file. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

4. Submit Lab Questionnaire on Gradescope

Remember to submit the lab questionnaire too.

If you see a question you do not know how to answer, ask TAs/UCAs in your lab, or use piazza, or go to one of our office hours.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.